

Chapitre 4 – Android - Bases de données

Sommaire

I.	Base de données interne	1
II.	Exemple d'application Android avec une base interne SQLite	1
III.	Base de données externe.....	6
IV.	Exemple d'application Android avec une base externe MySQL	7

I. Base de données interne

1. SQLite

Android possède un moteur de base de données nommé *SQLite*. Ce logiciel open-source est conçu pour :

- Limiter au maximum les tâches d'administration de la base de données,
- Etre exécuté dans le même processus que l'application de l'utilisateur.

Cela signifie que contrairement à MySQL ou SQL Server, il n'y a pas ici de serveur de base de données, ni d'application serveur. L'application Android utilise une bibliothèque (très légère : 300 Ko) comme il le fait pour d'autres fonctions.

Ce moteur est le plus distribué au monde (plus d'un milliard d'utilisations recensées) car il est indépendant de la plateforme. Il permet au sein d'un seul fichier, de stocker les structures des tables, leurs données, et les index.

Il n'intègre pas les primitives GRANT et REVOKE. Cette sécurité est à gérer via le système de fichier, globalement.

Son inconvénient :

- Pendant que la requête s'exécute, l'application ne réagit plus,
- La base ne se prête pas au partage, car le verrou se fait globalement sur le fichier.

Il convient cependant pour des utilisations très limitées.

II. Exemple d'application Android avec une base interne SQLite

1. Objectifs

La base gardera dans une table la liste des communes. L'application Android devra proposer la recherche d'une commune par rapport à son nom (saisi incomplètement).

- Maquetter l'interface Android,
- Ecrire l'application Android.

2. Application Android

Interface Android

Eléments de l'interface

Aperçu dans *Idea*

Aperçu de l'exécution



Code :

- Mise en place de l'activité, comme d'habitude, mais un petit plus : On garde un objet associé à la base *SQLite*, que l'on instancie au démarrage de l'activité :

```
private MaBaseSQLite maBaseSQLite;
private EditText etDebut;
private TextView tvResultat;

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.activity_main);
    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.LLMain), (v, insets) -> {
        Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
        return insets;
    });

    maBaseSQLite = new MaBaseSQLite(this);

    etDebut = findViewById(R.id.etDebut);
    tvResultat = findViewById(R.id.tvResultat);
}
```

On remarque que le nom de la base n'est pas indiqué ...

- Sur le clic, récupérer la saisie, appeler une méthode spécifique de la base qui fera la recherche, et ajouter son résultat au *TextView* :

```
public void rechercher (View v) {
    String debut = etDebut.getText().toString();
    String resultat = maBaseSQLite.communesDebutantPar(debut);
    tvResultat.append("Communes débutant par " + debut + " :\n" + resultat + "\n");
}
```

- Créer une classe Java supplémentaire dans le projet, nommée *MaBaseSQLite*. Modifier la déclaration de la classe *MaBaseSQLite* pour la faire descendre de *SQLiteOpenHelper* (moyen simple pour manipuler une base *SQLite*).

```
public class MaBaseSQLite extends SQLiteOpenHelper {
```

- On définit le constructeur de cette classe, qui appelle le constructeur parent, avec un nom pour la base et un numéro de version (nous le définissons à 1 ici) :

```
public MaBaseSQLite(@Nullable Context context) {
    super (context, "bdCommunes", null, 1);
}
```


- e. Implémenter les méthodes virtuelles de la classe parent (*onCreate* et *onUpgrade*). Le système vérifie si la base *bdCommunes* existe, et appelle la méthode *onCreate* s'il ne la trouve pas. Si elle existe, mais dans une version antérieure, c'est *onUpgrade* (voir ci-dessous). A nous de les surcharger pour créer nos tables, nos données, nos index, ...

```
@Override
public void onCreate(SQLiteDatabase db) {
    db.execSQL("CREATE TABLE communes ("
        + "id INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT, "
        + "nom TEXT(30) NOT NULL);");
    String[] lesCommunes = {"Paris", "Marseille", "Lyon", "Toulouse", "Nice", "Nantes",
        "Strasbourg", "Montpellier", "Bordeaux", "Lille"};
    for (String uneCommune : lesCommunes) {
        db.execSQL("INSERT INTO communes (nom) VALUES (\\"" + uneCommune + "\");");
    }
}
```

A noter :

- Les données sont conservées dans un tableau. Une boucle du type « for each » permet de les ajouter.
 - Sensibilité à l'injection SQL... pas gênant dans ce cas spécifique car les données sont connues.
 - Utiliser l'IA pour obtenir un jeu d'essai plus conséquent, avec une requête du genre : *générer en java un tableau de chaine nommé lesCommunes avec 50 villes*
- f. *SQLite* permet de gérer (voir le constructeur) un numéro de version de la base de données. En cas de différence détectée, il appelle la méthode *onUpgrade* pour permettre une mise à niveau. Pour l'instant, rien à faire :

```
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
}
```

- g. Il reste à écrire la méthode de recherche, s'appuyant sur le début de la commune en paramètre, et basée sur une requête SQL. Le résultat de la fonction contiendra les communes énumérées avec un tiret, sur des lignes différentes (voir ci-dessus *Aperçu de l'exécution*).

```
public String communesDebutantPar (String debut) {
    String res = "";

    SQLiteDatabase db = getWritableDatabase();
    Cursor cursor = db.rawQuery("SELECT nom FROM communes WHERE nom LIKE \\"" + debut + "%\"",
        null);
    if (cursor.moveToFirst()) {
        do {
            res += cursor.getString(0);
        } while (cursor.moveToNext());
    }

    return res;
}
```

Remarques :

- 1. Cette méthode intègre des aspects métiers (tirets et \n) dans la couche modèle, ce qu'il faut éviter.
- 2. Sensibilité à l'injection SQL : Problématique, à résoudre

Solutions :

- 1.a. Rendre une liste de communes (sans formatage) dans la couche modèle,
- 1.b. Adapter la couche métier en y mettant le formatage,

- 2. Passer le paramètre SQL dans la méthode *rawQuery* (comme *bindParam* en PHP)

```
public ArrayList<String> communesDebutantPar (String debut) {
    ArrayList<String> res = new ArrayList<>();

    SQLiteDatabase db = getWritableDatabase();
    Cursor cursor = db.rawQuery("SELECT nom FROM communes WHERE nom LIKE ?", new String[] {
debut + "%" });
    if (cursor.moveToFirst()) {
        do {
            res.add(cursor.getString(0));
        } while (cursor.moveToNext());
    }

    return res;
}
```

```
public void rechercher (View v) {
    String debut = etDebut.getText().toString();
    ArrayList<String> resultat = maBaseSQLite.communesDebutantPar(debut);
    tvResultat.append("Communes débutant par " + debut + " :\n");
    for (String nom : resultat) {
        tvResultat.append(" - " + nom + "\n");
    }
    tvResultat.append("\n");
}
```

Evolution : Compléter le nom par un *id* et un *code postal*

Passer à une version 2 du modèle, pour que les applications installées fassent l'évolution :

```
public MaBaseSQLite(@Nullable Context context) {
    super (context, "bdCommunes", null, 2);
}
```

Adapter *onUpgrade*. La solution ici est brutale : Supprimer la table pour la recréer avec les nouveaux champs. Dans un autre contexte, utiliser Alter table... :

```
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    db.execSQL("drop table communes");
    onCreate(db);
}
```

Le nouveau *onCreate* crée la nouvelle table. Pour cela, requêter l'IA avec ce type de requête : *générer en java un tableau avec noms et codes postaux nommé communes avec 50 villes*

Dans le modèle, utiliser une classe pour chaque table. Créer la classe *Commune* ainsi, à partir de la réponse de l'IA :

```

public class Commune {
    private long id;
    private String nom;
    private String codePostal;

    // Constructeur
    public Commune(String unNom, String unCodePostal) {
        id = 0;
        nom = unNom;
        codePostal = unCodePostal;
    }

    public Commune(long unId, String unNom, String unCodePostal) {
        id = unId;
        nom = unNom;
        codePostal = unCodePostal;
    }

    // Getters
    public long getId() {
        return id;
    }

    public String getNom() {
        return nom;
    }

    public String getCodePostal() {
        return codePostal;
    }
}

```

Remarques :

- Pas de setter car l'application ne propose pas de modifier ou saisir les données.
- Deux constructeurs, l'un pour remplir la table (*id* inconnu à ce moment là), l'autre pour fournir les communes commençant par ... (*id* connu)

Adapter *onCreate* en utilisant cette classe :

```

@Override
public void onCreate(SQLiteDatabase db) {
    db.execSQL("create table communes(" +
        "id integer primary key autoincrement," +
        "nom text(30)," +
        "cp text(5))");
    Commune[] communes = {
        new Commune("Paris", "75000"),
        new Commune("Lyon", "69000"),
        ...
        new Commune("Sète", "34200"),
        new Commune("Cergy", "95000")
    };
    for (Commune commune: communes) {
        db.execSQL("insert into communes (nom, cp) values (?, ?)", new String[] {
            commune.getNom(), commune.getCodePostal() });
    }
}

```

Adapter la méthode du modèle retournant une liste de communes :


```

public ArrayList<Commune> communesDebutantPar (String debut) {
    ArrayList<Commune> res = new ArrayList<>();

    SQLiteDatabase db = getWritableDatabase();
    Cursor cursor = db.rawQuery("SELECT id, nom, cp FROM communes WHERE nom LIKE ?", new
String[] { debut + "%" });
    if (cursor.moveToFirst()) {
        do {
            res.add(new Commune(cursor.getLong(0), cursor.getString(1), cursor.getString(2)));
        } while (cursor.moveToNext());
    }

    return res;
}

```

Adapter la méthode *rechercher* de la couche métier :

```

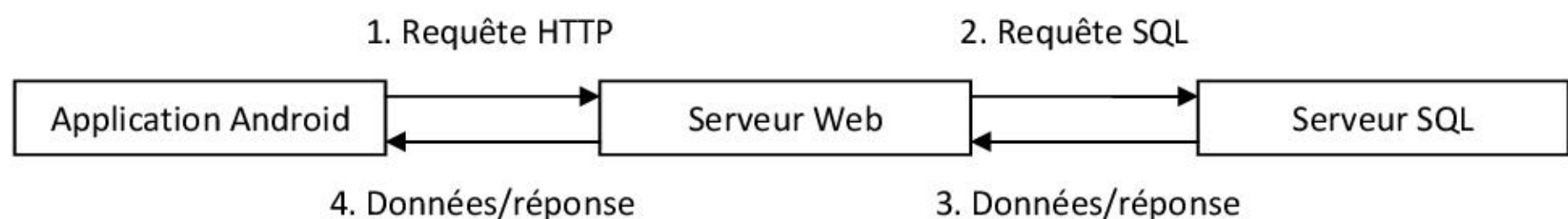
public void rechercher (View v) {
    String debut = etDebut.getText().toString();
    ArrayList<Commune> resultat = maBaseSQLite.communesDebutantPar(debut);
    tvResultat.append("Communes débutant par " + debut + " :\n");
    for (Commune commune : resultat) {
        tvResultat.append(" - " + commune.getNom() + " (" + commune.getCodePostal() + ")\n");
    }
    tvResultat.append("\n");
}

```

III. Base de données externe

1. Logique applicative

Pour partager les données, le modèle client/serveur est nécessaire. L'accès au serveur de base de données se fait indirectement pour des raisons de sécurité, via un serveur web.



On parle dans ce cas d'un **service web** assuré par le serveur, car l'URL demandée par l'application Android **ne délivre pas une page HTML, mais simplement des données**.

2. Formats d'échange des données

Chaque échange demande d'utiliser un format commun entre l'émetteur et le récepteur du flux.

- Flux 1 : La requête http contient l'URL et les paramètres éventuels (POST/GET)
- Flux 2 et 3 : Les échanges serveur Web et serveur SQL suivent un protocole qui ne concerne qu'eux. Il est imposé par le serveur SQL (le serveur Web contient donc un client SQL correspondant) : SQL Server/IIS ou MySQL/Apache.
- Flux 4 : C'est l'échange qui nous intéresse. Comment encoder et décoder l'information ?

a. Le format HTML

C'est le format à éviter, car trop orienté « affichage ».

b. Le format texte brut

Le plus « léger », mais entraîne des limitations (non conçu pour les jeux de caractère avec accents et cédille ...). Il n'est pas non plus adapté pour des données « à géométrie variable » (des lignes variables dans leur contenu). Il sera

cependant utilisé dans l'exemple car les données sont très peu structurées (tous les enregistrements sont semblables, avec une seule colonne).

c. Le format XML

Remédiant au problème précédent, il est en général très utilisé.

Exemple :

```
<menu id="file" value="Fichier">
  <popup>
    <menuitem value="Nouveau" onclick="CreateNewDoc()"/>
    <menuitem value="Ouvrir" onclick="OpenDoc()" />
    <menuitem value="Fermer" onclick="CloseDoc()" />
  </popup>
</menu>
```

d. Le format JSON

JSON (JavaScript Object Notation) est une alternative à XML, un peu plus légère (données moins volumineuses), qui permet de transformer en une chaîne tout type d'expression, y compris des tableaux, des objets, ... C'est un format très utilisé pour communiquer avec un serveur Web, et sera utilisé dans l'exemple.

Exemple :

```
{ "menu": {
  "id": "file",
  "value": "Fichier",
  "popup": {
    "menuitem": [
      { "value": "Nouveau", "onclick": "CreateNewDoc()" },
      { "value": "Open", "onclick": "OpenDoc()" },
      { "value": "Close", "onclick": "CloseDoc()" }
    ]
  }
}
```

3. Les éléments à mettre en place

Outre la base de données qui doit être en place, il faut développer :

- Le script PHP,
- Le programme Android l'utilisant.

IV.Exemple d'application Android avec une base externe MySQL

On va reprendre l'exemple précédent, avec la recherche d'une commune par son début.

- Créer dans la base *communes* de MySQL la table *commune* (*id*, *nom*).
- Table *commune* :

#	Nom	Type	Interclassement	Attributs	Null	Défaut	Extra
1	id	int(11)			Non	Aucune	AUTO_INCREMENT
2	nomVille	varchar(100)	latin1_swedish_ci		Oui	NULL	

```
CREATE TABLE IF NOT EXISTS commune (
  id int(11) NOT NULL AUTO_INCREMENT,
  nom varchar(100) DEFAULT NULL,
  PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```

- Insertion des données : Utiliser le script *Insert.sql* pour gérer les 35000 communes de France

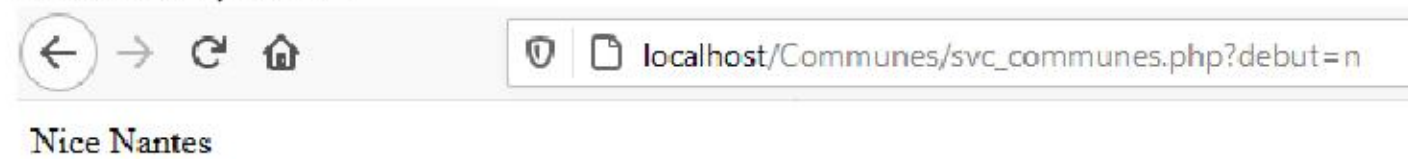
4. Script PHP version « texte brut » :

On parcourt le résultat de la requête, et chaque commune est écrite sur une ligne différente.

```
if (!isset($_GET["debut"])) die("Param absent");
try {
    $pdo=new PDO('mysql:host=127.0.0.1;dbname=communes;charset=utf8mb4','root','root',array(PDO::ATTR_ERRMODE =>
PDO::ERRMODE_EXCEPTION));
    $stmt=$pdo->prepare("SELECT nom FROM commune WHERE nom like :debut LIMIT 100", array(PDO::ATTR_CURSOR =>
PDO::CURSOR_FWDONLY));
    $stmt->bindValue(":debut", $_GET["debut"]."%", PDO::PARAM_STR);
    $stmt->execute();

    $lignes = $stmt->fetchAll();
    foreach ($lignes as $ligne) {
        echo ($ligne[0] . "\n");
    };
} catch (Exception $e) {
    die("Erreur fatale : " . $e->getMessage());
}
```

Test du script PHP :



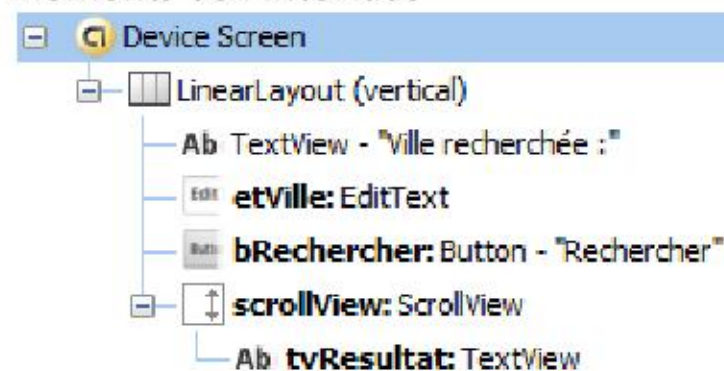
Données sources reçues :

```
1 Nice
2 Nantes
3
```

5. Application Android

Interface Android

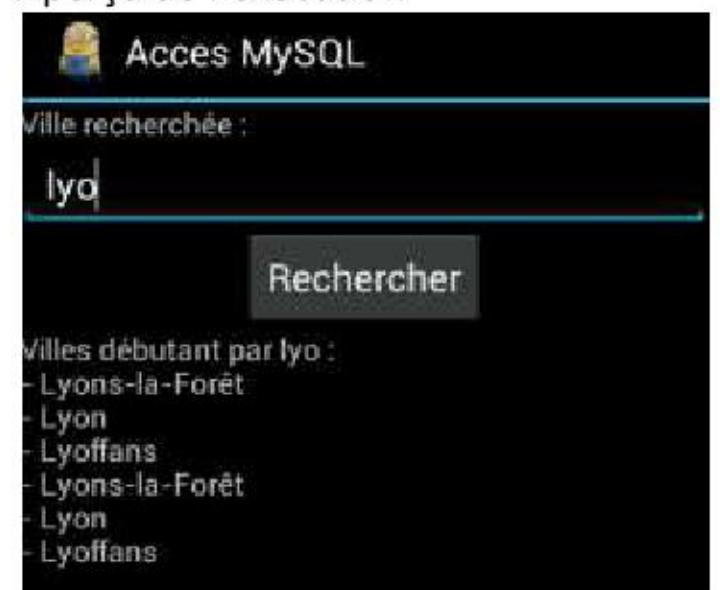
Eléments de l'interface



Aperçu dans Idea



Aperçu de l'exécution



Code :

- a. Mise en place de l'activité, comme d'habitude :

```
public class MyActivity extends Activity {
    /**
     * Called when the activity is first created.
     */
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main);
    }
}
```

- b. Sur le clic, récupérer la saisie, appeler le service web et ajouter son résultat au *TextView* :


```

public void clic(View view) {
    EditText ville;
    TextView resultat;
    String urlServiceWeb;

    ville = (EditText) findViewById(R.id.etVille);
    resultat = (TextView) findViewById(R.id.tvResultat);

    urlServiceWeb = "http://192.168.0.10/Villes/svc_villes.php?debut=" + ville.getText();
    resultat.setText("Villes débutant par " + ville.getText() + " :\n");
    resultat.append(getServerDataTexteBrut(urlServiceWeb));
}

```

Note : Pas de *localhost* ou *127.0.0.1* dans l'URL car les machines sont bien différentes !

- c. Définir la méthode *getServerDataTexteBrut* prenant l'URL du service web en paramètre et rendant le texte à afficher :

```
private String getServerDataTexteBrut(String urlString)
```

Cette méthode a besoin d'accéder au serveur web via le réseau, hors depuis quelques versions, Android interdit cette pratique de cette manière : Le thread principal doit toujours être disponible, hors l'accès au réseau peut l'immobiliser.

Deux possibilités :

- Créer et utiliser au autre thread,
- Utiliser un moyen pour autoriser l'accès réseau par le thread principal.

Le moyen le plus efficace est le premier, mais le plus simple est le second. Voici comment le mettre en œuvre :

```

// Autoriser les opérations réseau sur le thread principal
StrictMode.ThreadPolicy policy = new StrictMode.ThreadPolicy.Builder().permitAll().build();
StrictMode.setThreadPolicy(policy);

```

- d. Accéder au réseau demande des permissions particulières, qu'il faut intégrer. Ajouter dans le nœud *manifest* du fichier *AndroidManifest.xml* ceci :

```
<uses-permission android:name="android.permission.INTERNET" />
```

- e. Permettre l'accès réseau sans HTTPS dans les versions récentes d'Android :

- Modifier le *manifest* (nœud *application*) :

```

android:networkSecurityConfig="@xml/network_security_config"
android:usesCleartextTraffic="true"

```

- Créer le fichier de ressources *xml/network_security_config* :

```

<?xml version="1.0" encoding="utf-8"?>
<network-security-config>
    <domain-config cleartextTrafficPermitted="true">
        <domain includeSubdomains="true">172.16.47.51</domain>
    </domain-config>
</network-security-config>

```

- f. Retour au Java. Grâce à une connexion http, créer la requête http, exécuter cette requête, obtenir la réponse puis en extraire le contenu :

```

InputStream is = null;
...
URL url = new URL(urlString);
URLConnection connexion = (URLConnection)url.openConnection();
connexion.connect();
is = connexion.getInputStream();

```

- g. Quatre bulles font référence à du code non sécurisé :


```

URL url = new URL(urlString);
URLConnection connexion = (URLConnection)url.openConnection();
connexion.connect();
is = connexion.getInputStream();

```

Unhandled exception: java.io.IOException

- h. Ces exceptions sont à intercepter grâce à une construction *try ... catch ...*

```

try {
    ...
} catch (Exception expt) {
    Log.e("log_tag", "Erreur pendant la récupération des données : " + expt.toString());
};

```

- i. Une fois le contenu obtenu, lire ligne après ligne et placer l'ensemble dans la variable *ch* :

```

String ch = "";
...
BufferedReader br = new BufferedReader(new InputStreamReader(is));
String ligne;
while ((ligne = br.readLine()) != null) {
    ch = ch + "- " + ligne + "\n";
};

```

- j. Quand le résultat est créé, le passer comme résultat de la fonction :

```
return (ch);
```

- k. Voici donc la fonction *getServerDataTexteBrut* en intégralité :

```

private String getServerDataTexteBrut(String urlString) {
    InputStream is = null;
    String ch = "";

    // Autoriser les opérations réseau sur le thread principal
    StrictMode.ThreadPolicy policy = new StrictMode.ThreadPolicy.Builder().permitAll().build();
    StrictMode.setThreadPolicy(policy);

    try {
        URL url = new URL(urlString);
        URLConnection connexion = (URLConnection)url.openConnection();
        connexion.connect();
        is = connexion.getInputStream();

        BufferedReader br = new BufferedReader(new InputStreamReader(is));
        String ligne;
        while ((ligne = br.readLine()) != null) {
            ch = ch + "- " + ligne + "\n";
        };
    } catch (Exception expt) {
        Log.e("log_tag", "Erreur pendant la récupération des données : " + expt.toString());
    };
    return (ch);
}

```

6. Utilisation du format d'échange JSON

Script PHP

Les données doivent être encodées dans ce format :

```

if (!isset($_GET["debut"])) die("Param absent");
try {
    $pdo=new PDO('mysql:host=127.0.0.1;dbname=communes;charset=utf8mb4', 'root', 'root', array(PDO::ATTR_ERRMODE =>
PDO::ERRMODE_EXCEPTION));
    $stmt=$pdo->prepare("SELECT nom,cp FROM commune WHERE nom like :debut LIMIT 100", array(PDO::ATTR_CURSOR =>
PDO::CURSOR_FWDONLY));
    $stmt->bindValue (":debut", $_GET ["debut"]."%", PDO::PARAM_STR);
    $stmt->execute();

    echo (json_encode($stmt->fetchAll(PDO::FETCH_ASSOC)));
} catch (Exception $e) {

```

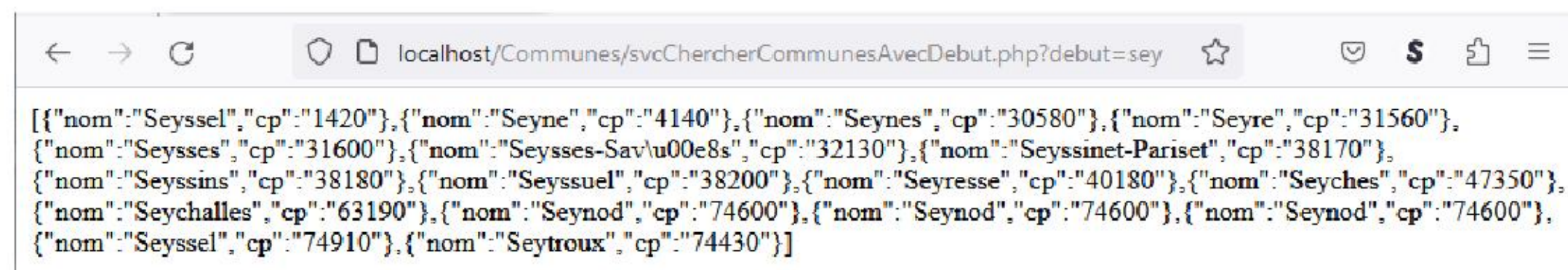


```

    die ("Erreur fatale : " . $e->getMessage());
}

```

Test du script PHP :



Modification du code de l'application Android :

La méthode `getServerDataJson` est similaire à `getServerDataTexteBrut` (la dupliquer) : Récupérer les données du serveur. On reconstruit ensuite la chaîne JSON dans `ch`, puis on crée l'objet `JSONArray` à partir de cette chaîne.

A noter : Toutes les données sont accessibles en même temps (pas de parcours séquentiel). Chaque ligne du tableau est stockée dans un objet `JSONObject`, qui permet d'obtenir une colonne par son nom. La chaîne `ch` listant les communes est construite et renvoyée.

```

...
BufferedReader br = new BufferedReader(new InputStreamReader(is));
String ligne;
while ((ligne = br.readLine()) != null) {
    ch = ch + ligne + "\n";
}
} catch (Exception e) {
    Log.e("log_tag", "Erreur pendant la récupération des données : " + e.toString());
}
// Parse les données JSON
try {
    JSONArray jArray = new JSONArray(ch);
    ch = "";
    for (int i = 0; i < jArray.length(); i++) {
        JSONObject jsonData = jArray.getJSONObject(i);
        ch = ch + " - " + jsonData.getString("nomVille") + "\n";
    }
} catch (JSONException e) {
    Log.e("log_tag", "Erreur pendant l'analyse des données : " + e.toString());
}
return (ch);

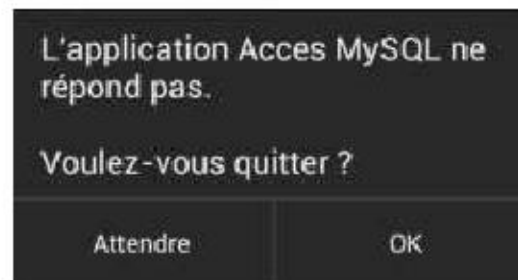
```

Aperçu de l'exécution :



7. Créer un thread pour l'accès réseau

Le système gère l'interface utilisateur uniquement par le thread principal (dénommé ThreadUI). Monopoliser ce thread provoque un blocage temporaire de l'application. Vérifier cela avec le cas précédent en arrêtant le serveur web : Le bouton reste coloré et un message arrive après 8 secondes environ :



Android propose plusieurs moyens, dont le plus simple est d'utiliser la classe *AsyncTask*.

Cette classe permet en surchargeant ses méthodes, de gérer ces événements :

- *onPreExecute* s'exécute au début, dans le threadUI,
- *doInBackground* est dans le thread spécifique (il contient la tâche longue),
- *onProgressUpdate* est dans le threadUI. Elle sert à actualiser une progression (voir documentation).
- *onPostExecute* est exécutée à la fin par le threadUI.

Ici seuls *doInBackground* et *onPostExecute* seront nécessaires.

Classe *TacheAsynchrone* descendant de *AsyncTask*

Le type du paramètre de *onPostExecute* est défini à *String* pour contenir les données à afficher :

```
private class TacheAsynchrone extends AsyncTask<Void,Integer,String> {  
  
    @Override  
    protected String doInBackground(Void... voids) {  
        return (getServeurDataJSON(urlServiceWeb));  
    }  
  
    @Override  
    protected void onPostExecute(String s) {  
        tvResultat.append(s);  
    }  
}
```

Cette classe nécessite que les variables *urlServiceWeb* et *tvResultat* placées initialement dans *onCreate* aient comme portée la classe *MyActivity* :

```
public class MainActivity extends Activity {  
    private String urlServiceWeb;  
    private TextView tvResultat;
```

L'utilisation de la classe *TacheAsynchrone* se fait sur le clic :


```

public void clic(View view) {
    EditText ville;

    ville = (EditText) findViewById(R.id.etVille);
    resultat = (TextView) findViewById(R.id.tvResultat);

    urlServiceWeb = "http://192.168.0.14/ville.php?ville=" + ville.getText();
    resultat.append("\nVilles débutant par " + ville.getText() + " :\n");

    TacheAsynchrone tacheAsynchrone = new TacheAsynchrone();
    tacheAsynchrone.execute();
}

```

Plus besoin de passer outre les règles d'Android :

```

// Autoriser les opérations réseau sur le thread principal
StrictMode.ThreadPolicy policy = new StrictMode.ThreadPolicy.Builder().permitAll().build();
StrictMode.setThreadPolicy(policy);

```

8. Utilisation d'un package simplifiant l'appel de services web : Volley

En lieu et place de la tâche asynchrone, on peut utiliser une librairie disponible sous forme de package.

Pas de try/catch, mais une fonction de rappel en cas d'erreur, tout comme une fonction de rappel dans le cas Ok :

```

StringRequest request = new StringRequest(Request.Method.GET, urlString,
    reponse -> Toast.makeText(this, "Réponse : " + reponse, Toast.LENGTH_SHORT).show()
    ,
    erreur -> Toast.makeText(this, "Erreur : " + erreur.getMessage(), Toast.LENGTH_SHORT).show()
);
Volley.newRequestQueue(this).add(request);

```

Le package s'ajoute dans le fichier *gradle* du module (partie *dependencies*) :

```
implementation("com.android.volley:volley:1.2.1")
```

Oui, mais attention !

Tous les packages disponibles sur GitHub (ou ailleurs) peuvent poser des problèmes de sécurité (portes dérobées notamment). Voir la vidéo *BackDoor* (Sem 2 - B3 - Web\Vidéo YouTube)

Utilisation de Volley vue en détail dans le chapitre suivant